ZK 专家养成课程 · 实战篇（L07–L09）

> 本卷由 3 讲合并装订而成（内容与单文件版逐字一致）。篇内各讲以 `# S/L/M xx` 标题分界，总纲见 `README_课程总纲.md`。

本卷目录

- **L07** · L07 · 代码精读路线图
- **L08** · L08 · 练习与项目集（P1~P12）
- **L09** · L09 · 进阶专题、研究前沿与资料清单

L07 · 代码精读路线图

> 学习目标：把前六讲的理论映射到本仓库的每一处实现；建立自己的调试与对拍工具；达到"给我一个协议问题，我能在 30 分钟内定位到代码"的水准。 前置：基础篇 S01–S06 + 提高篇 S07–S10（sumcheck / 承诺 / 收口 / 查表 / 系统全景）。预计 2 周（W9-W10）。

---

0. 环境与运行（第一天完成）

```
cargo test -p zkml            # 单元/集成测试（testing.rs: Pcs = HyperKZG<Bn254>）
cargo run -p bench-llm ...    # LLM 端到端 bench（模型缓存 model_cache/）
```

- 目录地图：`zkml/src/iop/`（证明协议层）、`layers/`（算子）、`lookup/`（查表）、`poly_commit/`（承诺封装）、`model/`（trace/量化模型）、`dp-crypto`（外部依赖：sumcheck 引擎、HyperKZG、transcript——源码在 cargo git checkout，改 deps 的 `branch` 可指到本地 fork）；
- 两把尺子：`measure.rs` 的打点（`layerwise_*`/`prove_commitment_opening` 等）输出耗时分布；`tracing` 的 debug 日志给出每 chunk/每节点事件。

1. 精读顺序（总原则：先骨架后血肉，每步带一个"读后必答"问题）

阶段	读什么	读后必答
A 骨架	`iop/prover.rs:588 prove_chunk` 通读（不进子函数）	八个步骤分别在哪几行？三个参数各来自哪里？
B 数据底座	`lookup/context.rs:233 generate_lookup_witness_for_chunk`；`poly_commit/context.rs:58 batch_commit`	多重度公式？batch 切块的阈值与上限、为什么限 128？
B' 引擎	dp-crypto `sumcheck/prover.rs:116`	Phase1/Phase2 分界在哪？轮多项式怎么归约？claimed sum 谁提供？
C 逆拓扑循环	prover.rs:717-856	`claims_for_node` 的两个来源？非可证明层如何透传？
C' einsum	`layers/einsum/prove.rs:46` + `axis.rs:478/1142`	Outer/Stacked/Contracted 判定的代码位置？终值三分类在哪几行？
C'' 查表层	`generic_prove.rs:66` → `inputs/proving.rs:26` → `logup_gkr/prover.rs:91`+`circuit.rs`	linking sumcheck 把哪两个"世界"焊起来？LogUp 每轮的挑战三元组？
D 收口	`poly_commit/prover.rs:97/140`；`poly_commit/mod.rs:83 build_sumcheck_expression`	eq_point_per_chunk 截掉的是什么位？RLC 之后还剩几条 claim？
E 打开	dp-crypto `arkyper/mod.rs:329 open / :386 verify`	三点 [r,−r,r²] 各自的作用？t=3 特化省了什么？
F 验证侧	`verifier.rs:721 verify / :755 verify_streaming / :715 一致性检查`	verify 与 verify_streaming 的 seed 差异？边界承诺相等比较的键是什么？

每阶段配的系统主教材章节：A→§四/五；B→§5.2；C→§5.5；C'→§5.5.3；C''→§5.5.4+OP-13；D/E→§5.8+OP-11；F→§7.3。

2. 调试与对拍工具箱（P9 的素材）

1. **transcript 吸收字节 dump**：在 `Transcript::append_bytes`（trait 默认实现处）加条件日志/写文件——这是端到端失败时的终极二分工具（FPGA 指导 §5.1）；
2. **claim 流 dump**：在 `Claim` 的构造点与 `add_witness_claim`/`add_common_claims` 打点，序列化 (poly_id, point, eval)；
3. **中间表 dump**：`generate_lookup_witness_for_chunk` 末尾、`fix_axes` 输出、sumcheck 每轮消息（`prove_round_and_update_state_parallel` 的返回值）；
4. **golden 对拍**：同一输入跑两份实现，逐字节比对上述 dump——硬件/优化改动的标准验收（S06 的 P2/P3 与 L08 的项目都依赖它）；
5. **小模型复现**：`Model::random_with_rng`（iop/mod.rs 测试）可用任意小模型跑全协议，是快速迭代的首选。

3. 关键约定速查（读代码时手边常备）

- 位序：表索引位 b ↔ 点坐标 r[b]；`fix_low` 相邻对；`evals` 倍增移位（FPGA 指导 §2.1/OP-04 有推导）；
- claimed sum 两种约定（einsum 重算 vs LogUp 先吸收）；
- transcript 标签总表（FPGA 指导附录 A）——任何新代码路径先登记；
- `read_challenges(n)` 连续同值的实现现状（blake3 XOF 位置语义）；
- chunk 内两份 transcript：块级（`b"model_chunk_proving"` + seed）与全局（seed 链）。

4. 常见卡点与解法（历届踩坑地图）

症状	根因	排查
verify 失败但各 sumcheck 单测通过	transcript 吸收顺序/字节不一致	dump 吸收流二分
手改位序后小例子"碰巧"通过	例子太对称（全 0/1 系数）	用非对称随机小例子 + 与 CPU 对拍

症状	根因	排查
多 claim 合并后值对不上	eq_point_per_chunk 的块选择位没截	检查点长度 vs num_chunks
证明耗时暴涨	请求了过大 witness 尺寸的 SRS/表未切块	看 batch_commit 的块数日志
streaming 模式 seed 错位	chunk 完成顺序与吸收顺序不一致	对照 `verify_streaming` 的循环顺序

5. 自测题

1. 从 `prove_chunk` 出发，默写 A→F 的调用树（到函数级），与仓库实测对照。
2. 解释为什么 `iop/mod.rs:118` 的 `From<Vec<ChunkProof>>` 如此简单——跨 chunk 的密码学绑定在哪三个机制里?
3. 给定一段新增代码会向 transcript 吸收一个 u64，你需要同步修改哪些地方?（答：两侧同函数重放处 + 附录 A 登记。）

6. 延伸阅读

- `prove_chunk_原理与实现详解.md`（第二遍精读，重点核对每个行号）;
- `prove_pipeline_FPGA算子开发指导.md`（把每个算子当"协议作用"词典用）;
- dp-crypto 的 `sumcheck/`、`arkyper/` 源码（第一手引擎实现）。

L08 · 练习与项目集（P1~P12）

> 使用方法：每个练习有明确的输入/产出/验收标准。完成后提交 `homework/`，导师批改。P1 对应 S02–S04，P2/P3 对应 S06，P4 对应 S10，P5/P6 对应 S07，P7 对应 S08/S09，P8–P10 对应 W8–W10，P11/P12 为毕业项。 通用验收：代码可 `cargo test` 通过；数值结果与 CPU 对拍逐位一致；附一页 README 说明思路。

P1 · 纸面数学（W1，验收：全对）

1. p = 101：域化 −7；表 [5,0,1,3] 在点 (2,3) 的 MLE 求值（写出 EQ 表全项）。
2. 证明：eq 表元素和恒为 1。
3. 用 Schwartz–Zippel 估算：d=6、挑战熵 128 bit（挑战 ∈ $[0,2^{128})$，见 S05）时 sumcheck 单轮健全性误差；再算 n=20 轮合计，并与"误用全域 2^{254}"的错误答案对比，解释两者差 2^{126} 倍的原因。

P2 · toy sumcheck（W2，验收：三情形）

实现 P/V 两侧 + 简化 transcript（blake3）。要求：

- ▢诚实证明通过；篡改表中一值必拒绝；篡改某轮多项式必拒绝（写两个负例测试）；
- ▢与 S06 §2–3 的 mod-13 手算跑例逐位一致（挑战固定为 7、4；以及换挑战 r_1=3、r_2=9 的作业版本）；
- ▢复杂度自证：n=16 单表（度 1 无乘法）全程读 ≈ $2\cdot(2^{17}-2)$ ≈ 26 万项 ≈ 8.4 MB——纯带宽操作，debug 构建毫秒级；若你的实现显著慢于该量级，先查是否做了多余拷贝。

P3 · sumcheck 工程化（W3，验收：逐位一致）

在 P2 基础上：二表乘积单项式；重心外推补点（max_degree 可配）；两阶段并行（线程数 P 可配）。

- ▢P>1 与 P=1 结果逐位一致；
- ▢一页对比：n=20 时 P=1/4/8 的耗时。

P4 · GKR 推导题（W4，书面）

S10 §2 的 claim 链推导（含 transcript 采样点标注）；选做：git 历史中 v1 版 L03 §6 的三道推导题。验收：claim 链图必须标注 transcript 采样点与挑战方向。

P5 · toy KZG（W5，对应 S07）

ark-bn254 上实现 commit/open/verify（自造 τ，N=2^8）。先在 mod-13 玩具上复现 S07 §5 的综合除法与"负例无商可除"，再上真实域。

- ▢配对等式核对通过；改 y、改表两个负例拒绝；
- ▢商多项式用综合除法（O(d)）实现并与朴素多项式除法对拍。

P6 · HyperKZG 三点打开（W6，对应 S08 §4）

- ▢编程验证 $2r \cdot p_{i+1}(r^2)$ 恒等式——先复现 S08 §4 的 mod-13 手算（r=5 与 r=3 两个版本），再上 N=2^{10} 随机 r、x；
- ▢实现折叠 + 三点求值 + 无余式商多项式 + 一次 batch MSM，与仓库 `HyperKZG::open` 的 (coms, w, v) 逐位一致；
- ▢一段话解释：为什么验证方不需要 p_i 在 r^2 的承诺。

P7 · toy LogUp（W7，对应 S09）

N=2^4 表、M 次查询的完整双侧 LogUp-GKR（复用 P3 引擎）。先复现 S09 §4–5 的手算（表 [1,2,3,4]、q=[2,1,1]、β=4），再泛化。

- ▢与 S09 §4–5 手算逐位一致；
- ▢作弊测试：查询含一个表外值 → 交叉核对必失败（S09 作业 2 的情形）；
- ▢2 列版本（α_col 合并）选做。

P8 · 系统数据流图 + 端到端（W8）

- ▢"稠密→GELU→稠密→logits"模型的全量证明数据流图（A4，标注承诺表/claim/采样点/代码行）；
- ▢GPT-2 端到端跑通：prove + `verify_streaming` 通过；记录（证明耗时、验证耗时、proof 大小、chunk 数）。

P9 · dump 工具与对拍（W9）

- ▢实现 transcript 吸收字节流 dump（开关控制）与 claim 流 dump；

- ▢同一输入跑两次（一次人为引入吸收顺序差异），展示二分定位到首个分歧点的过程（写进 README）；
- ▢用 dump 验证附录 A 标签表的完整性（发现新吸收点 → 更新该附录）。

P10 · 性能剖析（W10）

- ▢用 measure 打点输出 GPT-2 的分段耗时（witness 承诺 / sumcheck / 查表 / batching / 打开），与"MSM∝数据量、sumcheck∝节点数"两条规律对照；
- ▢找出当前最大的单一瓶颈并给出量化证据；
- ▢报告 ≤ 3 页。

P11 · 期末项目（W11，二选一）

A. 新算子接入：为本系统设计并实现一个新可证明算子（建议：SwiGLU 变体、max-pool 变体、RoPE）。要求：PIOP 设计文档（命题、协议、健全性论证）→ 实现 `ProvableOp`（prove + gen_lookup_witness + verify）→ 端到端测试通过 → 与 CPU 语义对拍。

B. mini-zkml：从零实现 200~500 行的迷你系统：两层稠密 + ReLU 的模型，PIOP（einsum sumcheck + 查表）+ toy PCS（可用 P5 的 KZG），支持 verify。要求结构对照 DeepProve（claim 链、batched sumcheck、一次打开）。

验收：设计文档（2 页）+ 可运行代码 + 测试 + 30 分钟答辩。

P12 · 论文精读报告（W12）

从以下选一篇：Gemini §2.4.2（HyperKZG 数学）、LogUp 论文、arXiv:2210.00264（distributed sumcheck）、DeepProve §4-5。报告含：问题、构造、健全性论证、与本系统的关系、若由你实现会改什么。

考核标准（毕业线）

项	权重	通过线
P1~P7 正确性	40%	全部验收项通过
P8~P10	25%	完整 + 有证据链
P11 项目	25%	端到端可运行 + 设计文档自洽
答辩	10%	能白板复述任一核心证明（导师抽签：sumcheck 健全性 / 三点打开引理 / LogUp 恒等式 三选一）

L09 · 进阶专题、研究前沿与资料清单

> 学习目标：建立"协议设计空间"的全景视角；知道本系统之外的路线上发生了什么；掌握读协议论文的方法。W12 + 毕业后持续使用。

---

1. 进阶专题（每个 1~2 小时的拓展阅读量）

1.1 折叠型方案与递归

sumcheck 的折叠思想被推广成一类"折叠型证明系统"：Nova/SuperNova（折叠式 IVC，多次证明累加成一次）、ProtoStar 等。与本系统的关系：HyperKZG 本身就是 Nova 生态的产物（仓库代码头注释：port 自 Nova provider）；若未来要做"证明的证明"（如把 chunk 证明递归聚合），折叠方案是首选方向。读法：先懂 Nova 的 relaxed R1CS 折叠直觉即可，不必先啃完整论文。

1.2 FRI 与 BaseFold（无 trusted setup 路线）

FRI 用哈希+ Merkle 树做多线性承诺（无配对、无 setup、抗量子），代价是证明更大、验证方做哈希重算。BaseFold 把 FRI 折叠与多线性 sumcheck 结合。DeepProve 论文 §5 的评测表同时含 KZG 与 BaseFold 变体——两条路线在本代码库中**PIOP 完全共享**（PCS 解耦）。思考题：若目标平台无 trusted setup 基础设施，迁移成本在哪?

1.3 分布式与流式证明

- **分布式**：ProverGraph 执行图（prover.rs:927 起）把 chunk 分派到多 worker；论文 §4.5 的多机方案；
- **流式**：本分支的核心——trace 逐步产生、chunk 证完即释放（`StreamingProver`）；seed 链替代公共挑战是协议级改动（对比 `Verifier::verify` vs `verify_streaming` 的 transcript 差异，这是你已精读的内容在真实演进中的活例子）。

1.4 硬件加速

prove 管线的计算收敛为三类核：MSM（Pippenger/GPU）、sumcheck 轮引擎（带宽型）、表级数据算子（EQ/折叠/RLC）。FPGA 指导文档给出了完整算子规格与集成设计——把它当作"协议的硬件投影"来读，你会对协议的算子结构有更深一层理解。

1.5 安全视角

- transcript 等价性的工程攻击面（吸收顺序、序列化格式、XOF 语义）——本项目附录 A 的存在本身就是安全工程；
- trusted setup 仪式（powers-of-tau）；
- 量化边界与溢出：证明系统证明的是"i64 计算的域像一致"，i64 溢出本身不在证明范围内——位宽预算是模型/编译期的责任（conv 的 output_bitsize 即此逻辑）。

2. 研究开放问题（可作长期课题）

1. **查表层 overhead 的压缩**：GELU 级查表 witness 占 witness 承诺量的大头，能否用更少的列/更小的表?
2. **注意力二次项**：score 张量使数据量随 s^2 涨——流式已解决内存，证明时间呢?（可结合 sparse attention 的证明友好结构。）
3. **PCS 混合**：witness 用 BaseFold（免 setup）、权重用 KZG（setup 一次）的混合承诺是否值得?
4. **验证方的进一步降载**：当前 V 需重放全部 sumcheck；对超深模型能否引入证明聚合?

3. 读协议论文的方法（导师方法论）

1. 先读 §问题定义与 Theorem 陈述（10 分钟），自问"这个定理正是我想要的吗"；

2. 跳到 Protocol 框图，手推一个 n=2 的例子（论文例题通常有）；
3. 回读健全性证明，找出"随机挑战在哪个等式上发力"；
4. 列出"工程落点清单"：承诺几次、sumcheck 几轮、挑战几个、验证方重算什么；
5. 用本仓库做对照：哪些已有、哪些是新原语。

4. 资料清单

书与讲义

- Justin Thaler《Proofs, Arguments, and Zero-Knowledge》—— Georgetown 主页免费全文；本课程必读章节：§2（背景）、§3.1（sumcheck 预备）、§4.1-4.2（sumcheck/GKR 完整推导）、§7（多线性承诺概览）。

论文（按标题/出处检索）

1. Lund–Fortnow–Karloff–Nisan, *Algebraic Methods for Interactive Proof Systems*（JACM 1992）—— sumcheck 起点；
2. Goldwasser–Kalai–Rothblum, *Delegating Computation: Interactive Proofs for Muggles*（STOC'08 / JACM'15）—— GKR；
3. Kate–Zaverucha–Goldberg, *Constant-Size Commitments to Polynomials and Their Applications*（2010）—— KZG；
4. *Gemini: Stretching Curve Proofs over Binomial Curves*（ePrint 2022/420）§2.4.2 —— HyperKZG 数学出处（代码头注释引用）；
5. *LogUp: Efficient logarithmic (lookup) arguments*（ePrint 2023，项目引作 [PH23]）与 Haböck《Multivariate lookups based on logarithmic derivatives》（ePrint 2022）—— 查表两篇；
6. arXiv:2210.00264 —— 两阶段并行 sumcheck（代码注释引用）；
7. *BaseFold*（FRI×多线性折叠）—— PCS 备选路线；
8. *zkCNN*（CCS 2021）—— FFT 矩阵委托出处（代码注释引用）；
9. **DeepProve 论文**——仓库 `docs/zkml_bootcamp/deepprove.pdf`，第一精读对象。

博客/视频（直觉来源）

- Vitalik Buterin 的 SNARK 系列博文（sumcheck、GKR、KZG 各有专篇，按标题检索）；
- ZK Whiteboard Sessions / ZK-Hack 系列视频（概念直觉，选看承诺与查表两集）。

本仓库教材

- `prove_chunk_原理与实现详解.md` —— 系统主教材；
- `prove_pipeline_FPGA算子开发指导.md` —— 算子规格与全局约定速查；
- DeepProve 论文中文对照（若有）与 `docs/zkml_bootcamp/` 其余文档按需参考（以源码为准）。

5. 毕业之后的路标

- 读源码新分支的 RFC/变更（如流式协议审计的 W1/W2/W3 绑定点演进）；
- 给系统提一个改进 PR（新算子 / 性能 / dump 工具）——真实的维护者反馈是最好的老师；
- 关注 ePrint 上 "sumcheck" / "lookup argument" / "multilinear PCS" 的月度新增，用第 3 节的方法每周筛读一篇。